GitLab

Grundlagen und GitLab DevOps

Dr.-Ing. Grigory Devadze

Was Sie mitnehmen

- **Einordnen:** CI, Delivery und Deployment unterscheiden.
- **Zusammenarbeiten:** Änderungen mit Git und Merge Requests integrieren.
- **Automatisieren:** Pipelines gezielt auslösen und strukturieren.
- **Absichern:** Variablen, Daten und Runner-Rechte passend begrenzen.
- **Nachweisen:** Tests, Berichte und Container reproduzierbar bereitstellen.

Alle Teile arbeiten an derselben kleinen Änderung: Ein PowerShell-Skript soll Dateien **nach Namen sortiert** ausgeben. Am Ende ist nachvollziehbar, welcher Auftrag, welcher Code, welcher Test und welches Artifact zusammengehören.

Agenda

1. GitLab im Entwicklungszyklus
2. Git und Projektstruktur
3. Branching und Zusammenarbeit
4. Die erste Pipeline
5. Pipeline-Steuerung
6. Secrets und Pipeline-Daten
7. Runner, Container und Tests
8. Auto DevOps und nächste Schritte

Teil I · GitLab im Entwicklungszyklus

Lernziele Teil I

- Unterscheiden, was Git erledigt und was GitLab ergänzt.
- CI, Continuous Delivery und Continuous Deployment daran unterscheiden, **wer die Auslieferung freigibt**.
- Für eine Änderung festlegen, welcher Test sie absichert.

Zuerst klären wir, was das Skript künftig tun soll. Danach sehen wir, welche Werkzeuge dabei helfen.

Beispielanforderung: sortierte Dateiliste

Unser Kursprojekt heißt **Banana Project**. Sein Skript `ListFiles.ps1` gibt alle Dateien eines Verzeichnisses mit ihrer Größe aus.

Heute: Reihenfolge, wie das Dateisystem sie liefert

```
beta.txt - 2 Bytes
alpha.txt - 1 Bytes
```

Gewünscht: aufsteigend nach Dateinamen, Format unverändert

```
alpha.txt - 1 Bytes
beta.txt - 2 Bytes
```

Aufgabe des Teams:

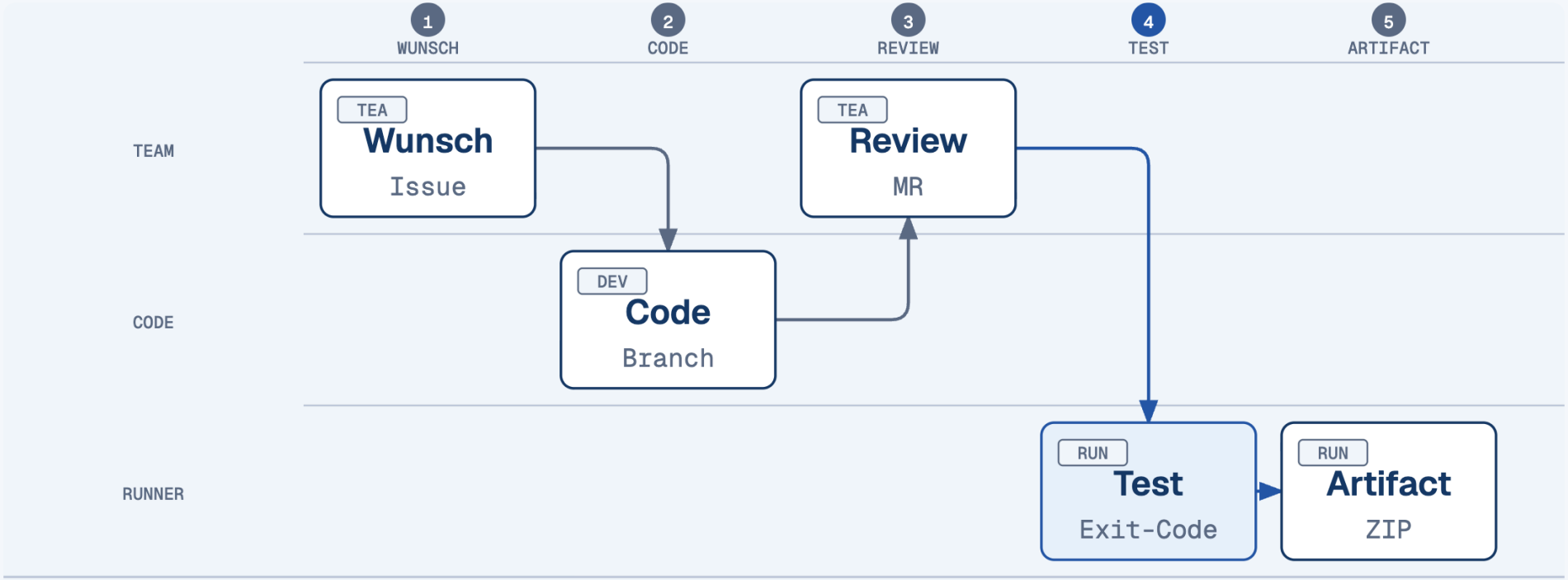
1. Skript ändern, sodass es nach Namen sortiert.
2. Test ergänzen, der die Reihenfolge prüft.
3. Geprüften Stand als Artifact übergeben.

Git und GitLab: Aufgabenteilung

Git	GitLab
Speichert Dateistände und ihre Historie.	Stellt das Repository dem Team bereit.
Läuft lokal, auch ohne Server.	Ergänzt Issues, Merge Requests und Pipelines.
Verwaltet Commits, Branches und Remotes.	Regelt Zugriffsrechte, Reviews und automatische Jobs.

Einen Commit können Sie lokal ohne GitLab anlegen. Issues und Review-Kommentare liegen dagegen nur in GitLab: `git clone` lädt sie nicht mit herunter.

Weg einer Änderung vom Issue zum Artifact



Jeder Schritt klärt eine Frage: **Brauchen wir das? Ist es verständlich? Sind die Tests grün? Darf es raus?** Review und Pipeline laufen oft mehrfach.

Bausteine eines GitLab-Projekts

Baustein	Beispiel im Banana Project
Issue	„Dateiliste nach Namen sortieren“
Repository	`ListFiles.ps1` und zugehörige Tests
Merge Request	Die Codeänderung mit den Review-Kommentaren
Pipeline	Ergebnisse von Lint, Test und ZIP-Erzeugung
Artifact	Das ZIP, das ein Job erzeugt hat

Das Repository speichert den Quellcode mit seiner Historie. Das Artifact ist **nur eine Datei, die ein Job daraus erzeugt hat.**

Continuous Integration

Definition: Das Team führt seine Änderungen oft, meist mehrmals täglich, im gemeinsamen Branch zusammen. Build und Tests laufen dabei automatisch und melden, ob noch alles funktioniert.

Kein CI: Ein Branch bleibt wochenlang getrennt und wird jeden Tag gebaut. Mit der Arbeit der anderen wird er aber erst spät zusammengeführt.

Lint, Test und Package

Prüfung	Was sie im Beispiel prüft	Was sie nicht prüft
Lint	Hält der PowerShell-Code die eingestellten Regeln ein?	Ist die Ausgabe richtig sortiert?
Test	Die Fälle, die im Test stehen	Fälle, die niemand geschrieben hat
Package	Liegen die vorgesehenen Dateien im ZIP?	Arbeitet das Skript fachlich richtig?

Ein grüner Haken sagt nur, dass diese eine Prüfung bestanden ist. **Was geprüft wurde, steht im Job**, nicht in seinem Namen.

Continuous Delivery

Definition: Build, Tests und ZIP-Erzeugung laufen automatisch. Am Ende liegt ein Stand bereit, der jederzeit ausgeliefert werden kann. Ob er in Produktion geht, **entscheidet ein Mensch**.

Im Beispiel: Das Artifact ist gebaut und getestet. Erst wenn eine berechtigte Person es freigibt, wird es in die Zielumgebung ausgeliefert.

Noch kein Deployment: Das ZIP liegt nur in GitLab. Auf dem Zielsystem hat sich nichts geändert.

Continuous Deployment

Definition: Jede Änderung, die alle Prüfungen besteht, geht automatisch in Produktion. Niemand gibt sie vorher noch einmal frei.

Der Unterschied zu Delivery: **Gibt vor der Produktion noch ein Mensch frei?** Ob ein Job `deploy` heißt, spielt dafür keine Rolle.

Im Beispiel: Fehlt der Test für die Sortierung, geht eine falsch sortierte Liste direkt in Produktion. Bei automatischer Auslieferung müssen die Tests deshalb alles prüfen, was sonst ein Mensch vor der Freigabe prüfen würde.

Delivery und Deployment im Vergleich

	Continuous Delivery	Continuous Deployment
Build und Tests laufen automatisch	Ja	Ja
Wer entscheidet über die Auslieferung?	Ein Mensch	Die Pipeline
Ergebnis am Ende	Artifact wartet auf Freigabe	Artifact läuft in Produktion

Aussagekraft eines grünen Tests

Angenommen, der Test legt nur eine Datei an und erwartet diese Ausgabe:

```
banana.txt - 6 Bytes
```

Der Test prüft Namen und Größe, aber **nicht die Reihenfolge**. Er bliebe auch grün, wenn das Skript absteigend sortiert.

Erst ein Test mit zwei Dateien erkennt eine falsche Reihenfolge. „Alle Tests erfolgreich“ heißt nur: Die Fälle, die jemand geschrieben hat, sind bestanden.

Entscheidungen und Verantwortliche

Frage	Wer entscheidet
Was soll sich ändern?	Auftraggeber bzw. Product Owner, zusammen mit dem Team
Ist die Änderung verständlich und richtig?	Reviewer
Welche Prüfungen müssen laufen?	Das Team; ein Maintainer baut sie in die Pipeline ein
Darf ausgeliefert werden?	Eine benannte Person, z. B. Product Owner oder Betrieb

GitLab legt diese Zuständigkeiten nicht fest, das **vereinbart das Team**. Eine Person kann mehrere davon übernehmen.

Rückblick Teil I

Issue → Änderung → Review → Test → Freigabe: An jedem Schritt prüft jemand etwas anderes.

Bei CI führt das Team oft zusammen und testet automatisch. Delivery und Deployment unterscheiden sich darin, ob vor der Produktion ein Mensch freigibt.

Im nächsten Teil geht es darum, **welche lokalen Dateien Git speichert und zu GitLab überträgt.**

Teil II · Git und Projektstruktur

Lernziele Teil II

- Gruppe, Projekt, Repository: Was liegt wo?
- Am Diff sehen: Working Tree, Index oder Commit?
- Änderungen auswählen, committen und zu GitLab pushen.
- Die Änderung in Issue, Label, Milestone und README beschreiben.

Beispiel bleibt die Sortierung in `ListFiles.ps1`. Git speichert, was sich an der Datei ändert. Issue und Co. sagen, **warum**.

Gruppe, Projekt, Repository

Ebene	Enthält	Beispiel
Gruppe	Mitglieder, Untergruppen, Projekte	`team`
Projekt	Repository, Issues, MRs, Pipelines	`team/banana`
Repository	Dateien und ihre Historie	`ListFiles.ps1`, Tests, README

Ein Projekt umfasst mehr als das Repository. Ihr lokaler Ordner darf anders heißen als der Pfad in GitLab.

Geerbte Mitgliedschaft

Beispiel: Jemand ist Developer in der Gruppe `team`. Das Projekt `banana` liegt in dieser Gruppe.

Was Sie ändern	Was passiert
Person direkt aus dem Projekt nehmen	Zugriff über die Gruppe bleibt
Mitgliedschaft in der Gruppe `team` ändern	Ändert den Zugriff über die Gruppe

Bei unerwarteten Rechten reicht der Blick ins Projekt nicht. Prüfen Sie **direkte, geerbte und geteilte Mitgliedschaften** einzeln.

Sichtbarkeit, Rolle und Branch-Regel

Frage	Regelt
Wer kann das Projekt sehen?	Sichtbarkeit und Mitgliedschaft
Was darf die Person tun?	Ihre Rolle in Gruppe oder Projekt
Darf sie direkt auf `main` pushen?	Schutzregeln für den Branch

Privat hält nur Außenstehende fern. Wer im Team was darf und wie Code nach `main` kommt, regeln Sie extra.

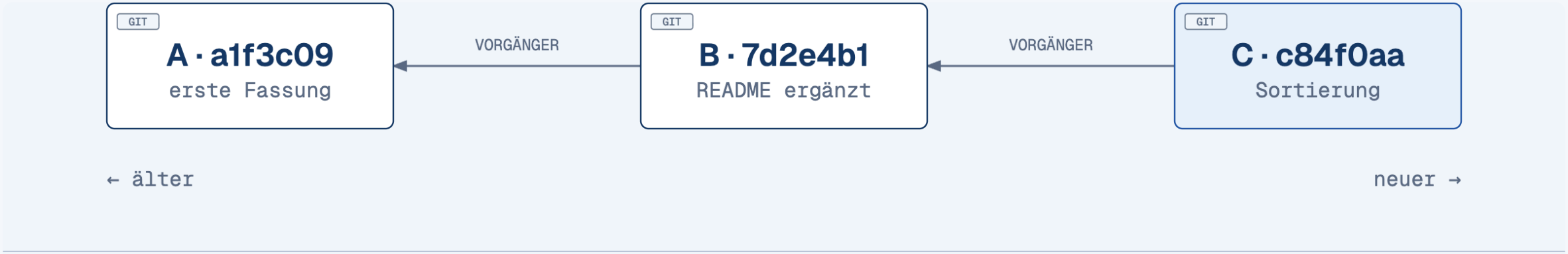
Rollen von Guest bis Owner

Rolle	Darf typischerweise
Guest	Mitdiskutieren; in privaten Repositories standardmäßig keinen Code sehen
Reporter	Code und Projekt lesen
Developer	Code ändern
Maintainer	Projekt einstellen, Branches schützen
Owner	Volle Kontrolle über Gruppe oder Projekt

Es gibt weitere Rollen und Sonderrechte. Ob eine Aktion erlaubt ist, regeln **Rechtetabelle, Ebene und Projektregeln**.

GitLab: Roles and permissions

Commit: Projektstand mit ID und Vorgänger

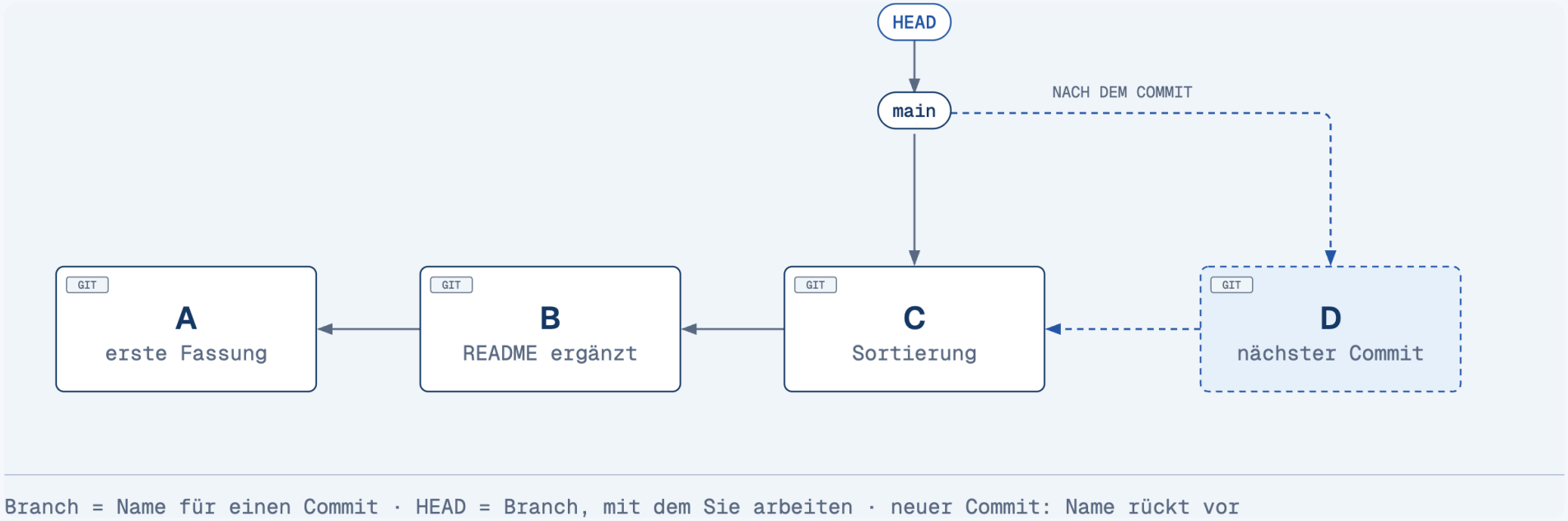


Jeder Commit kennt seinen Vorgänger · die ID gehört zu genau diesem Projektstand

Ein Commit speichert den Stand **aller Dateien, die Git verfolgt**, dazu Autor, Zeit, Nachricht und seinen Vorgänger. Danach ändert er sich nie mehr.

Aus diesem Inhalt berechnet Git die **Commit-ID**, hier auf sieben Zeichen gekürzt. Gleiche ID auf Ihrem Rechner und in GitLab heißt: gleicher Stand.

Branch und HEAD als Zeiger



Ein Branch ist ein **Name, der auf einen Commit zeigt**, hier `main` auf C. `HEAD` zeigt auf den Branch, mit dem Sie gerade arbeiten.

Ihr nächster Commit D bekommt C als Vorgänger, und `main` rückt auf D vor. A, B und C bleiben, wie sie sind.

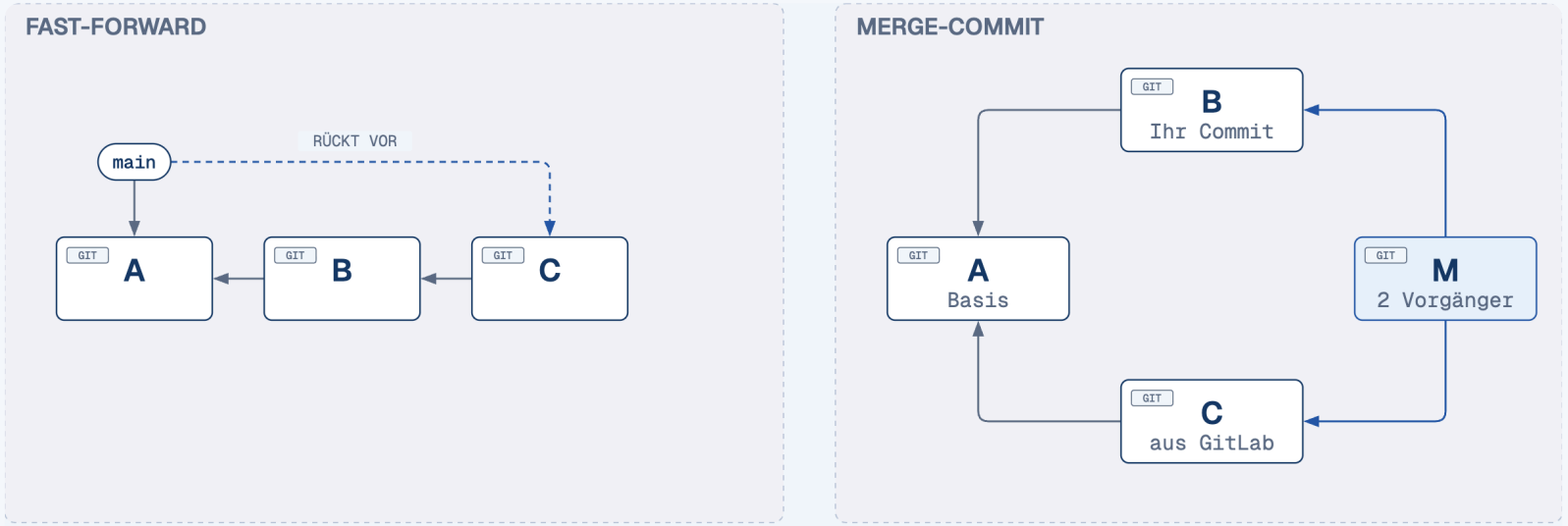
`origin/main`: Stand von GitLab auf Ihrem Rechner



`origin/main` ist Ihre **lokale Kopie** des Branches `main` in GitLab. Git aktualisiert sie nur, wenn Sie mit GitLab sprechen: bei `fetch`, `pull` und `push`.

Im Bild hat jemand B und C nach GitLab gepusht. Ihr Rechner weiß davon nichts, bis Sie den Stand holen.

Zusammenführen: Fast-forward oder Merge-Commit

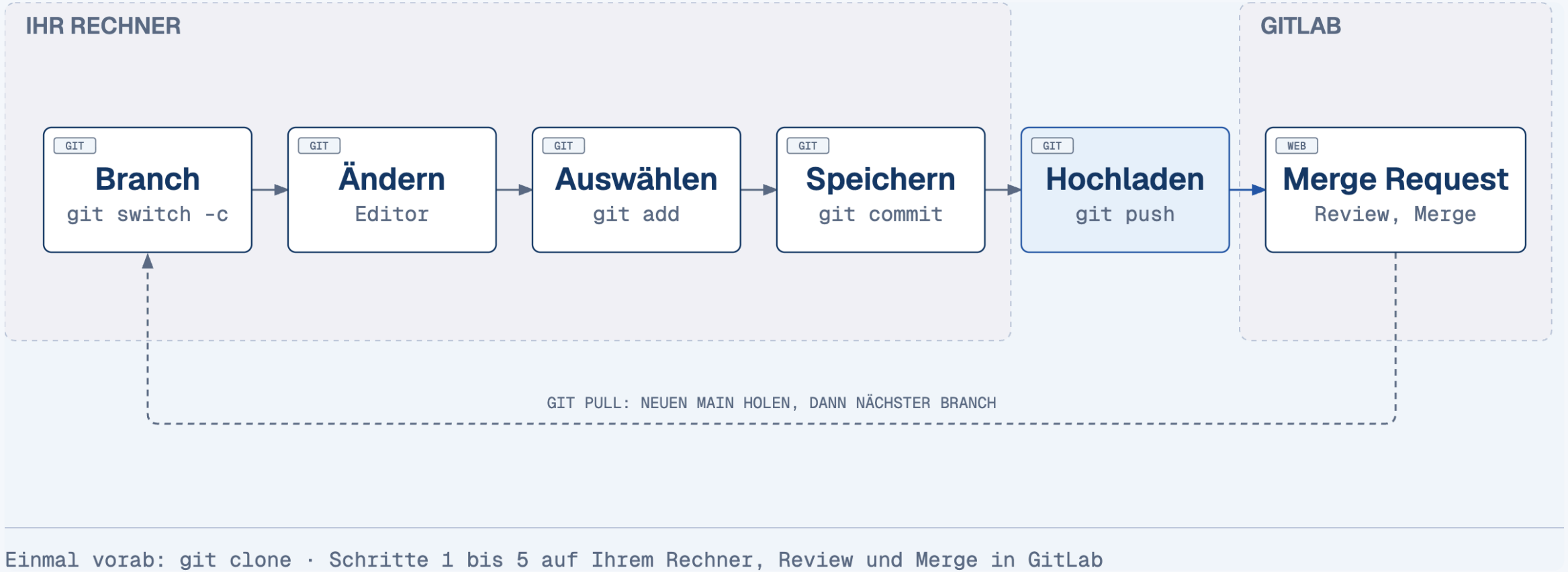


Fast-forward: nur der Name rückt vor, kein neuer Commit · Merge-Commit: verbindet zwei Linien

Hat Ihr Branch keine eigenen neuen Commits, rückt der Name einfach vor: **Fast-forward**. Nur das lässt `git pull --ff-only` zu.

Haben beide Seiten neue Commits, entsteht ein **Merge-Commit** mit zwei Vorgängern. Ändern beide dieselbe Stelle, gibt es dabei einen Konflikt.

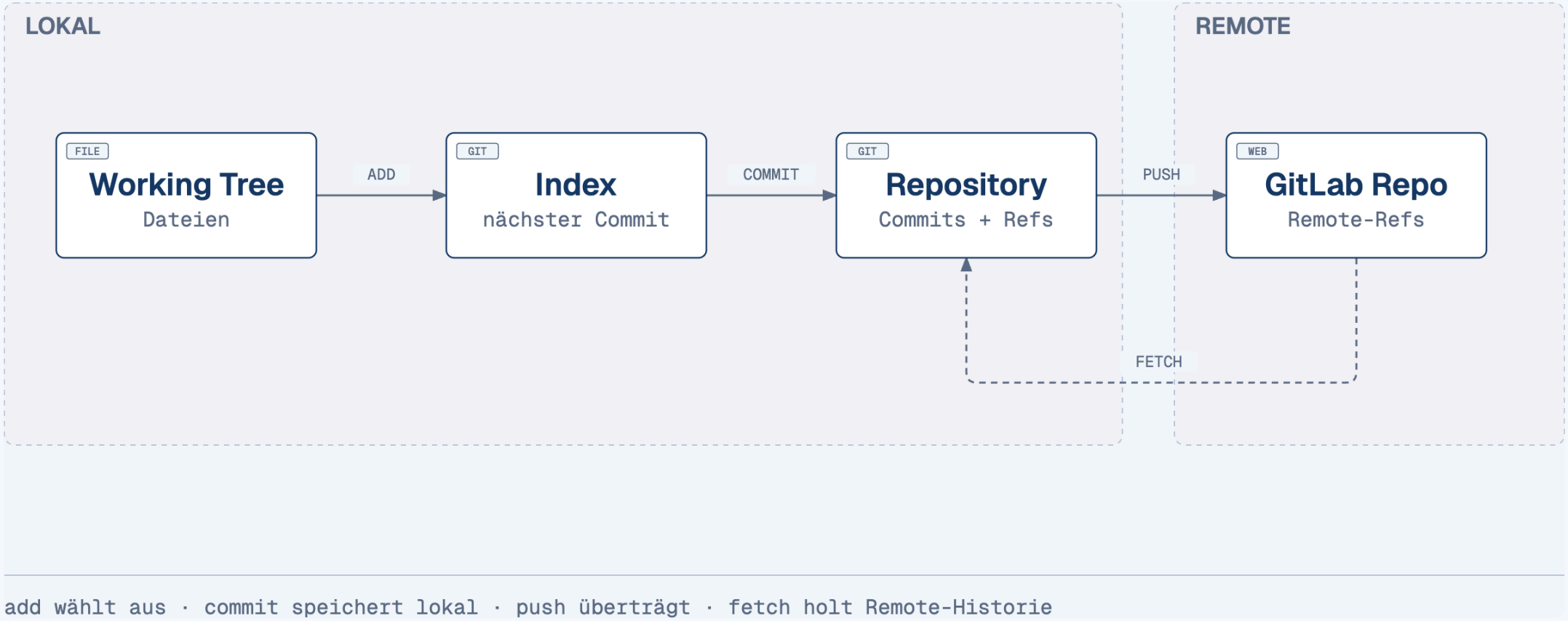
Arbeitszyklus mit Git und GitLab



Einmal vorab: git clone · Schritte 1 bis 5 auf Ihrem Rechner, Review und Merge in GitLab

Jede Änderung geht diesen Weg. Die folgenden Folien zeigen die Befehle dazu einzeln, zuerst `add` und `commit`.

Working Tree, Index und Commit



Ein Commit speichert nur, was im Index liegt. Änderungen nur im Working Tree fehlen darin. Bei GitLab ist noch nichts angekommen.

Diff der Sortieränderung

Die entscheidende Stelle in `ListFiles.ps1`:

```
 Get-ChildItem -LiteralPath $Path -File -ErrorAction Stop |
+    Sort-Object -Property Name |
     ForEach-Object { '{0} - {1} Bytes' -f $_.Name, $_.Length }
```

Welche Dateien und in welchem Format: bleibt gleich. Neu ist nur: **Das Skript sortiert nach Namen**.

Der Diff zeigt, was sich geändert hat. Ob die Ausgabe wirklich sortiert ist, muss ein Test zeigen.

`git status --short`: die zwei Spalten

Beispielausgabe (ohne Merge-Konflikt):

```
 M ListFiles.ps1
M  README.md
?? notes.txt
```

Spalte 1: Index vs. letzter Commit. **Spalte 2:** Working Tree vs. Index.

Die Änderung am Skript liegt noch nicht im Index, die am README schon. `notes.txt` kennt Git noch gar nicht.

```
git status --short
```

`git diff` und `git diff --staged`

```
git diff -- ListFiles.ps1
git diff --staged -- ListFiles.ps1
```

Befehl	Vergleich	Frage
`git diff`	Working Tree mit Index	Was fehlt noch im Index?
`git diff --staged`	Index mit letztem Commit	Was kommt in den nächsten Commit?

Nach `add` kann `git diff` leer sein, obwohl die Änderung **noch nicht committed** ist.

git add

```
git add ListFiles.ps1
git diff --staged -- ListFiles.ps1
git status --short
```

ListFiles.ps1	Vor add	Nach add
Kurzstatus	`M`	`M`
Sortierzeile liegt im Index	Nein	Ja

Einzeln hinzufügen hält Fremdes aus dem Commit. `add` legt die Datei **so, wie sie jetzt ist**, in den Index. Ändern Sie danach weiter, ist ein neues `add` nötig.

Index und Working Tree derselben Datei

Angenommen, Sie ändern die Sortierzeile nach `add` noch einmal:

Bereich	Sortierzeile
Letzter Commit	Noch keine Sortierung
Index	`Sort-Object -Property Name`
Working Tree	`Sort-Object -Property Name -Descending`

```
MM ListFiles.ps1
```

Ohne neues `add` speichert der Commit die **aufsteigende** Fassung. `-Descending` bleibt nur im Working Tree. Prüfen Sie deshalb vor dem Commit beide Diffs.

git commit

```
git commit -m "feat: sortiere Dateiliste nach Namen"
git log -1 --oneline
```

Der Commit speichert den Inhalt des Index, dazu Vorgänger, Autor und Zeit. Er liegt **nur lokal** und ist noch nicht bei GitLab.

Der Betreff sagt, was sich ändert, die Beschreibung bei Bedarf, warum. `feat:` ist eine Absprache im Team, Git verlangt das nicht.

Historie lesen: `log`, `show`, `diff`

```
git log --oneline -5
git show HEAD -- ListFiles.ps1
git diff HEAD~1 HEAD -- ListFiles.ps1
```

- `log` listet die Commits auf.
- `show` zeigt einen Commit samt Änderungen.
- `diff` vergleicht zwei Commits.

`HEAD` ist der Commit, auf dem Sie gerade stehen, `HEAD~1` sein Vorgänger. Fehlt der, etwa in einem neuen Repository, scheitert der Vergleich.

.gitignore

Beispiel für `.gitignore`:

```
dist/
TestResults/
git-summary.txt
```

Die Regeln wirken nur auf Dateien, **die Git noch nicht verfolgt**. Ein schon committeter Report bleibt drin, ein committetes Geheimnis bleibt in der Historie.

Die Pipeline kann ein ignoriertes ZIP trotzdem bauen und als Artifact weitergeben. Ins Repository muss es dafür nicht.

`git restore --staged`

```
git restore --staged ListFiles.ps1
git status --short
git diff -- ListFiles.ps1
```

Git nimmt die Datei wieder aus dem Index (zurück auf den letzten Commit). **Ihre Änderung im Working Tree bleibt.**

Vorsicht: `git restore ListFiles.ps1` ohne `--staged` überschreibt die Datei im Working Tree mit dem Indexstand. Was nicht im Index lag, ist weg.

`git clone`

Beispieladresse, nicht unser GitLab:

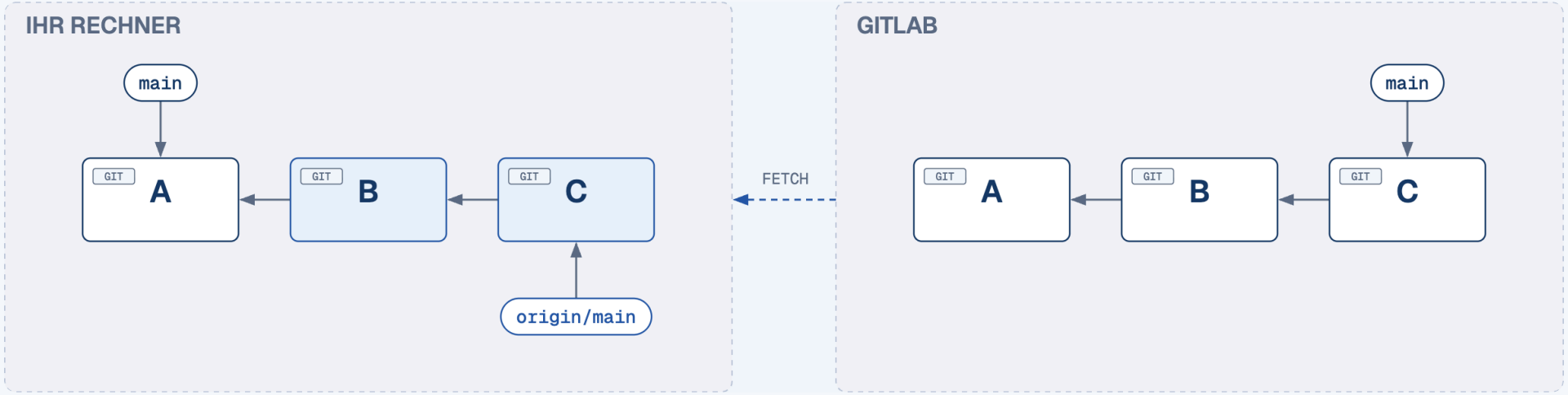
```
git clone https://gitlab.example.org/team/banana.git
cd banana
git remote -v
```

`git clone` lädt das Repository herunter und richtet `origin` als Remote ein. Branches und Historie kommen mit. Issues und Review-Kommentare bleiben in GitLab.

`origin` ist nur der übliche Name für ein Remote. Ob dort der offizielle Stand des Teams liegt, verrät der Name nicht.

git fetch

```
git fetch origin
git log --oneline HEAD..origin/main
```



`fetch` lädt B und C herunter und setzt `origin/main` auf C. Ihr `main` und Ihre Dateien bleiben auf A. `git log` listet die Commits, die in `origin/main` sind, in `HEAD` aber noch nicht.

`git pull --ff-only`

Wenn `main` keine offenen Änderungen hat und `origin` folgt:

```
git switch main
git pull --ff-only
```

`pull` holt Commits und baut sie ein. `--ff-only` aktualisiert nur, wenn Ihr `main` keine eigenen Commits hat, die im Remote fehlen.

Haben beide Seiten neue Commits, bricht der Befehl ab. Schauen Sie dann in die Historie und **wählen Sie Merge oder Rebase**, statt etwas zu erzwingen.

`git push`

Angenommen, Sie haben den Branch `feature/dateiliste` schon lokal:

```
git push -u origin feature/dateiliste
git branch -vv
```

Der erste Push legt den Branch in GitLab an. `-u` verknüpft ihn mit Ihrem lokalen Branch. Danach überträgt `git push` nur neue Commits.

Ein Push bringt nichts nach `main`. Dafür gibt es den vereinbarten Weg, zum Beispiel einen Merge Request.

Diagnose: Push abgelehnt

Meldung	Möglicher Grund	Was Sie prüfen
Authentifizierung abgelehnt	Falsches Remote oder falscher Account	Remote-URL und Anmeldedaten
`non-fast-forward`	Remote hat neue Commits	`fetch`, dann Historie vergleichen
Geschützter Branch	Kein Pushrecht für diesen Branch	Zielbranch und Schutzregel

Ein Force-Push hilft weder bei fehlenden Rechten noch bei inhaltlichen Fragen. Er kann Commits im Remote überschreiben: **keine Standardlösung**.

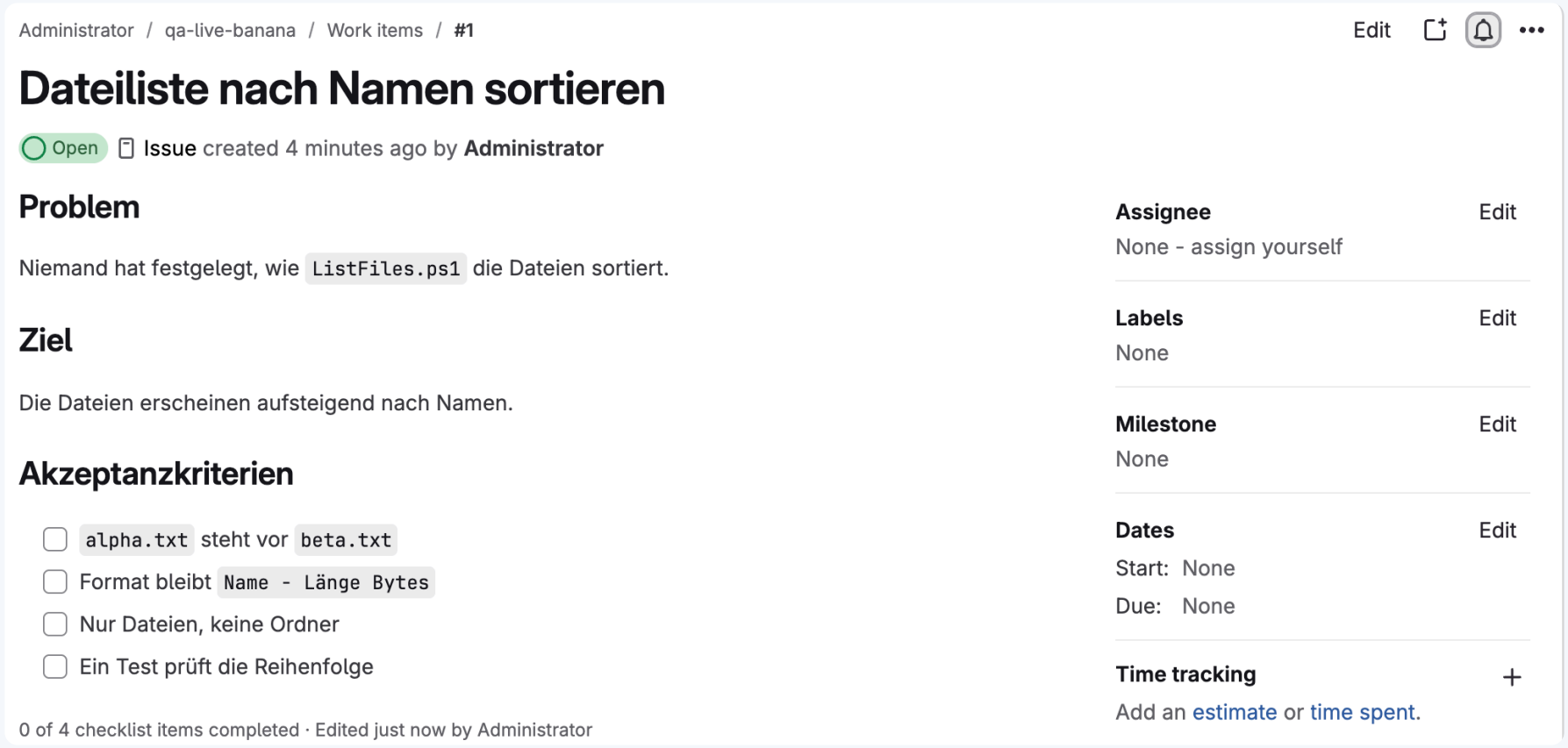
Web Editor und Web IDE

Wozu: Dateien direkt in GitLab ändern, ohne Clone und ohne lokales Git.

Web Editor	Web IDE
Einzelne Datei bearbeiten	Mehrere Dateien zusammen bearbeiten
Kleiner README-Fix	Code, Test und README gemeinsam ändern
Änderung committen	Änderungen auswählen und committen

Jede Änderung im Browser ist ein normaler Commit, mit Branch und Review wie lokal. Tests laufen dort nicht, sie starten erst in der Pipeline.

Issue zur Sortieränderung



Die Ausgabe war bisher nicht unbedingt falsch sortiert. Neu ist: **Die Reihenfolge ist jetzt festgelegt.**

Labels und Scoped Labels

Normales Label	Bedeutung bei uns
`bug`	Skript tut nicht, was es soll
`documentation`	Beschreibung oder Beispiele ändern
`ready-for-review`	Laut Teamabsprache bereit fürs Review

Die Namen legt das Team fest, GitLab gibt sie nicht vor. Ein Issue kann mehrere normale Labels tragen. Die gibt es in **Free**.

Scoped Labels wie `workflow::ready` schließen sich gegenseitig aus und brauchen **Premium / Ultimate**.

GitLab: Scoped labels

Milestones

Beispiel: „Dateiliste 1.0“

Issue im Milestone	Wozu
Sortierung festlegen	Klären, was das Skript tun soll
Skript und Test ändern	Sortieren einbauen und testen
README aktualisieren	Verhalten erklären

Ein Projekt-Milestone sammelt Issues eines Projekts, ein Gruppen-Milestone auch aus mehreren. Schließen Sie einen Milestone, **bleiben seine offenen Issues offen**.

Issue und Merge Request verknüpfen

Beispiel für die MR-Beschreibung:

```
Sortiert die Dateiliste aufsteigend nach Namen.
Closes #12
```

Das Issue sagt **warum**, der MR zeigt **wie**, dort läuft das Review.

`Closes #12` schließt das Issue, sobald der MR in den Default-Branch gemergt wird. Voraussetzung: Automatisches Schließen ist an, und wer mergt, darf das Issue schließen.

README des Banana Project

```
# Banana Project

ListFiles.ps1 gibt die Dateien des gewählten Verzeichnisses aus.
Jede Zeile enthält Dateiname und Größe in Bytes.
Die Ausgabe ist aufsteigend nach Dateinamen sortiert.
```

`-Path` gibt das Verzeichnis an. Ohne `-Path` nimmt das Skript `.`.

Steht die Sortierung nur im README, sortiert das Skript noch nicht. **README, Skript und Test** müssen dasselbe sagen.

Ein Thema pro Änderung

So soll `ListFiles.ps1` danach aussehen:

```
Get-ChildItem -LiteralPath $Path -File -ErrorAction Stop |
    Sort-Object -Property Name |
    ForEach-Object { '{0} - {1} Bytes' -f $_.Name, $_.Length }
```

Geändert wird nur die Reihenfolge. Format, Dateiauswahl und Fehlerbehandlung bleiben, wie sie sind.

Test und README zur Sortierung gehören dazu. Wollen Sie nebenbei umbauen, machen Sie dafür eine eigene Änderung: Sonst wird das Review mühsam.

Rückblick Teil II

Sie sehen	Offene Frage
Skript bearbeitet	Ist der richtige Stand im Index?
Commit lokal da	Ist er im richtigen Remote angekommen?
Code in GitLab	Sagen Issue, README und Test dasselbe?

Jetzt arbeiten mehrere mit: **Was passiert, wenn andere `main` ändern, während wir arbeiten?**

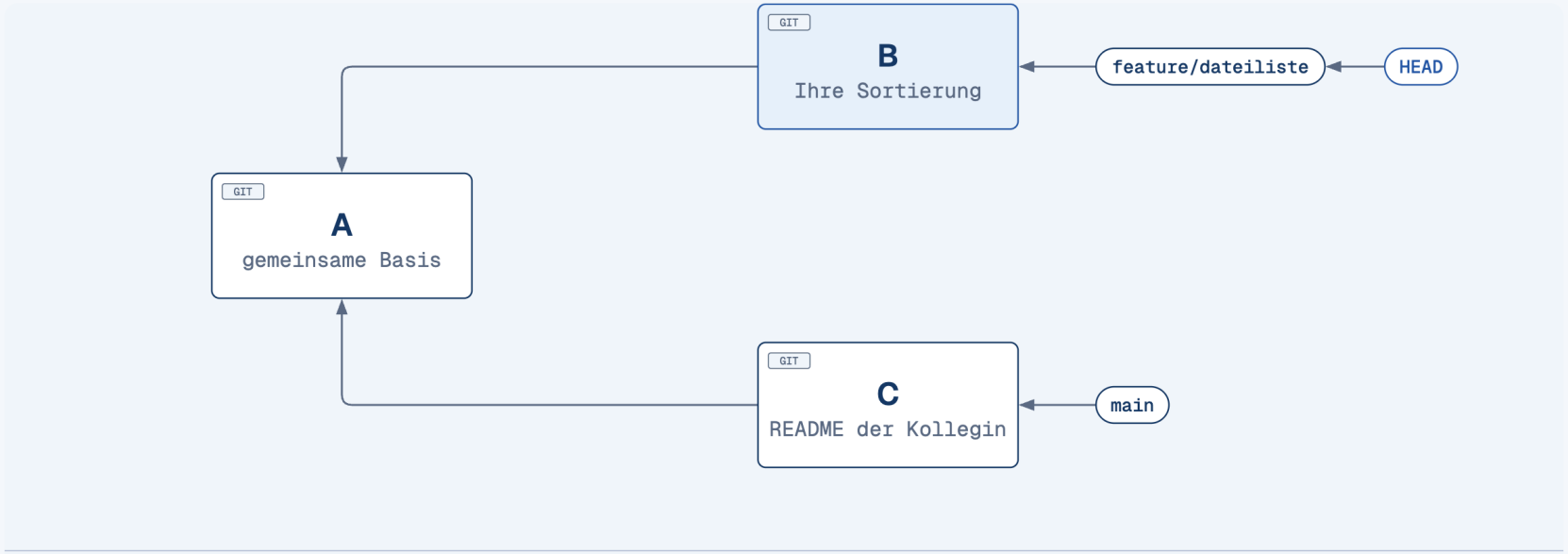
Teil III · Branching und Zusammenarbeit

Lernziele Teil III

- Ein Branch- und Release-Modell wählen, das zu Ihrem Projekt passt.
- Unterscheiden: Wer darf pushen, wer darf mergen, wer gibt fachlich frei?
- Einen Konflikt gemeinsam inhaltlich klären und auflösen.
- Die Sortieränderung reviewen: Was zeigen Test und Diff?

Ein Merge ohne Konflikt heißt nur: Git konnte die Änderungen zusammenführen. **Ob Text und Sortierung stimmen**, prüfen Sie selbst.

Zwei Branches mit gemeinsamer Vorgeschichte



Beide Namen zeigten zuerst auf A · HEAD = Branch, der im Working Tree ausgecheckt ist

Erst zeigen beide Namen auf A. Dann committen Sie die Sortierung B, eine Kollegin ergänzt auf `main` die README in C.

Im Working Tree sehen Sie den ausgecheckten Branch. Ein Branch legt **keinen zweiten Projektordner** an.

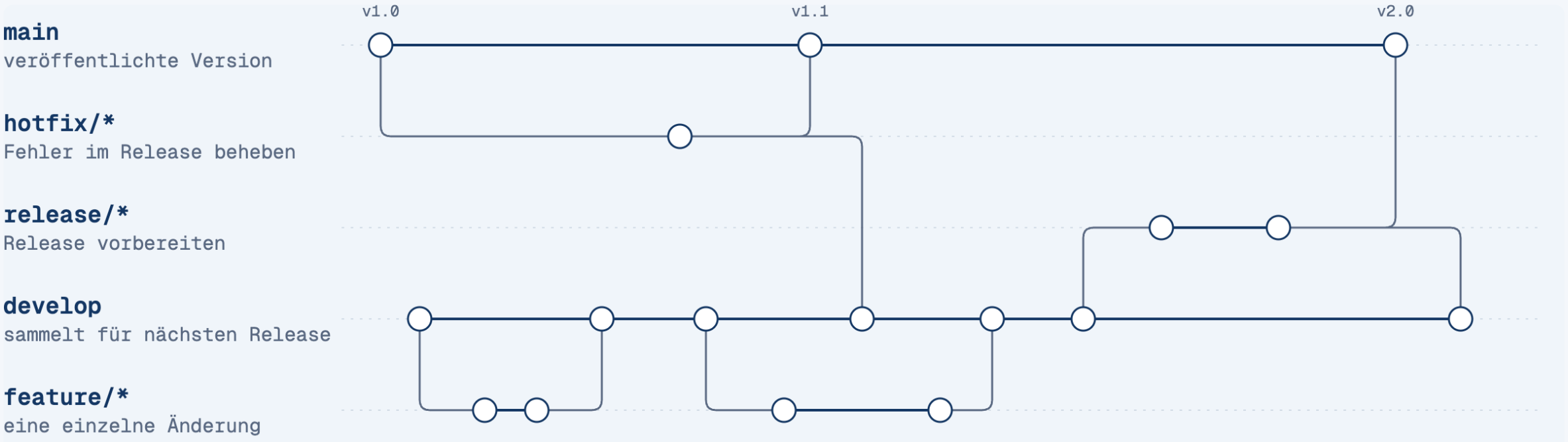
Feature-Branch anlegen: `git switch -c`

Sauber starten:

```
git switch main
git pull --ff-only
git switch -c feature/dateiliste
git branch --show-current
```

Damit starten Sie auf dem aktuellen `main`. Während Sie arbeiten, kommen dort neue Commits dazu. Im Feature-Branch liegt **nur Ihre Änderung**. Beim Zusammenführen kann es trotzdem später Konflikte geben.

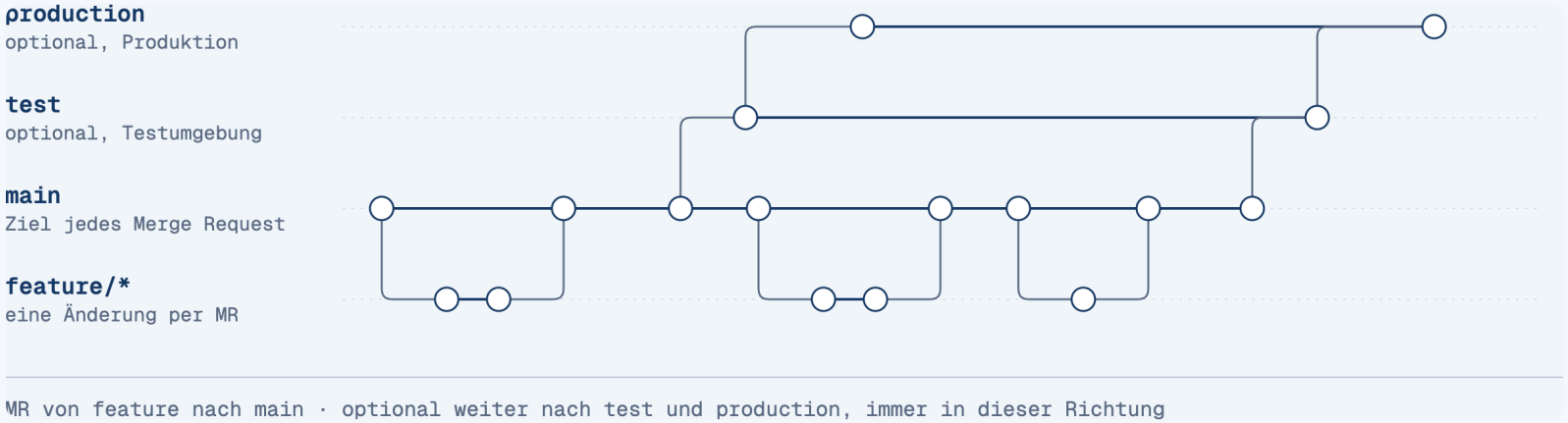
Git Flow



Features über develop · Release-Branch bereitet die Version vor · Hotfix von main zurück in main und develop

Git Flow ist eine **Absprache im Team**, keine Git-Funktion. Je mehr Branchtypen, desto mehr muss das Team regeln: Was wird wohin gemergt?

GitLab Flow

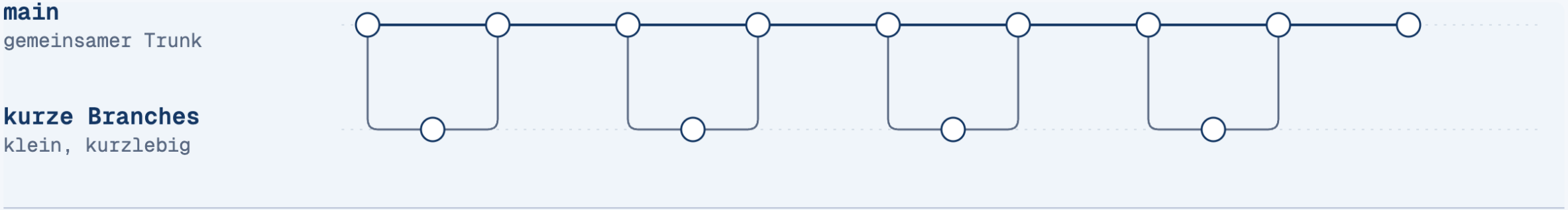


Feature-Branches zweigen von `main` ab. Ein Merge Request bündelt Code, Issue, Diskussion und Review.

Optional kommen weitere Branches hinzu. Für jeden legen Sie fest, was von wo nach wo gemergt wird:

- **Environment-Branches:** je ein Branch pro Umgebung (z. B. Test, Produktion).
- **Release-Branches:** mehrere Produktversionen parallel pflegen.

Trunk-Based Development



Kleine Änderungen zweigen kurz ab und kommen per MR mit Pipeline schnell nach main zurück

Alle mergen in einen gemeinsamen Branch, meist `main`. Änderungen bleiben klein, Feature-Branches leben nur kurz und werden schnell zurückgemergt.

Dafür braucht es automatische Tests und schnelles Review.

Trunk-Based heißt **nicht automatisch: direkt pushen ohne Prüfung**. Eine formale Freigabe passt auch zu kurzen Zyklen.

GitLab: Trunk-based development with short-lived branches

Auswahl der Branching-Strategie

Situation	Was das Team regeln muss
Mehrere Versionen gleichzeitig pflegen	Wege für Releases und Hotfixes festlegen
Eigener Stand pro Umgebung	Wer bringt was in welche Umgebung?
Kleine Änderungen häufig mergen	Review und Tests müssen schnell sein

Branching-Strategie, Merge-Methode und Freigabe für Produktion sind drei getrennte Entscheidungen.

Protected Branches: Einstellungen

Einstellung	Regelt
Allowed to merge	Wer über einen Merge Request mergen darf
Allowed to push and merge	Wer direkt pushen und mergen darf
Allowed to force push	Wer den geschützten Branch umschreiben darf

Branch-Schutz nach Rollen gibt es in **Free**. Ob Developer oder nur Maintainer mergen dürfen, steht in der Regel selbst. „Protected“ allein sagt das nicht.

GitLab: Protected branches

Direkten Push auf `main` sperren

Beispiel: Änderungen kommen nur per MR auf `main`.

Einstellung	Gewählter Wert
Allowed to merge	**Developers + Maintainers**
Allowed to push and merge	**No one**
Allowed to force push	Deaktiviert

„Nicht konfiguriert“ ist nicht dasselbe wie „No one“. Weitere Regeln (Wildcard, Gruppe) können zusätzliche Rechte geben. Entscheidend ist, was alle Regeln zusammen erlauben.

GitLab: Push and merge permissions

Parallele Änderungen und Merge-Konflikte

Ändern zwei Branches verschiedene Stellen, führt Git das meist selbst zusammen. Ändern beide dieselbe Stelle, müssen Sie entscheiden.

Beispiel: Beide Branches ersetzen dieselbe README-Zeile:

Feature-Branch	Inzwischen geänderter `main`
„Ausgabe nach Namen sortiert“	„Ausgabe enthält Größen in Bytes“

Beide Zeilen sind richtig. Git weiß nicht, was in der README stehen soll. Umgekehrt kann ein **Merge ohne Konflikt inhaltlich falsch** sein.

Konflikt im Feature-Branch: `git merge origin/main`

```
git switch feature/dateiliste
git fetch origin
git merge origin/main
git status
```

Mögliche Ausgabe:

```
CONFLICT (content): Merge conflict in README.md
```

Hier holen wir den aktuellen `main` **in den Feature-Branch.** `main` selbst bleibt dabei unverändert.

Konfliktmarker in der Datei

Bei diesem Merge steht `HEAD` für den aktuellen Feature-Branch:

```
<<<<<<< HEAD
Ausgabe nach Namen sortiert.
=======
Ausgabe enthält Größen in Bytes.
>>>>>>> origin/main
```

Zwischen den Markern steht der Konflikt. Nicht einfach eine Seite übernehmen, sondern fragen: **Was muss am Ende in der README stehen?**

Konflikt auflösen: `git add` und Commit

Gemeinsam vereinbarter README-Text:

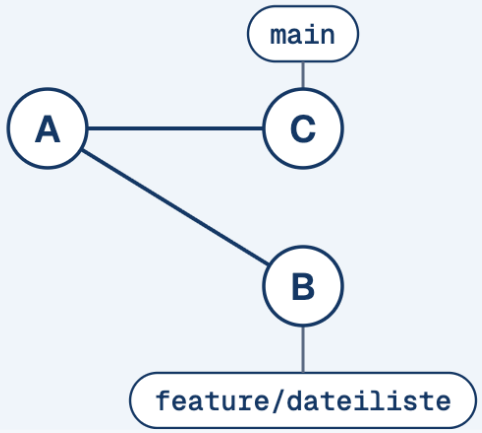
```
Ausgabe aufsteigend nach Namen sortiert;
Größen werden in Bytes angegeben.
```

```
git add README.md
git diff --staged
git commit -m "merge: integriere aktuellen main"
```

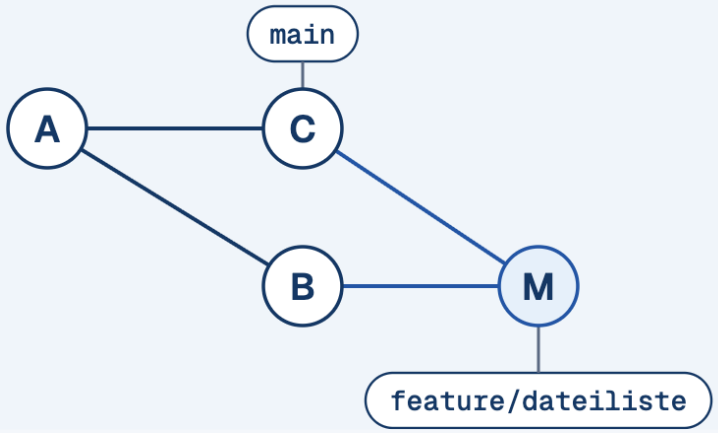
`add` markiert den Konflikt als gelöst. Ob Text und Skript stimmen, prüft Git nicht. **Tests und Review müssen zeigen, dass beides erfüllt ist.**

Merge und Rebase im Vergleich

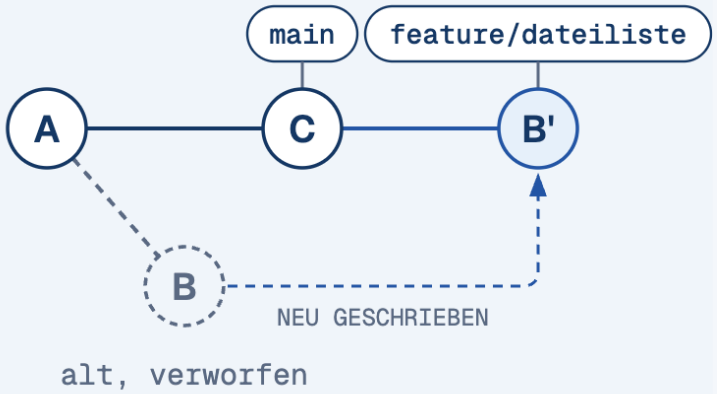
Vorher
zwei Branches ab A



Merge
git merge main



Rebase
git rebase main



Merge behält die Historie, Rebase schreibt sie neu. Schon gepushte Commits schreiben Sie nur um, **wenn das Team das abgesprochen hat.** Andere könnten darauf aufbauen.

Merge Request: Source, Target, Diff, Review

Für unsere Änderung:

Feld	Bedeutung
Source	`feature/dateiliste` mit der Sortieränderung
Target	`main`, dorthin soll sie
Diff	Was sich gegenüber `main` ändert
Review	Diskussion und Entscheidung zur Änderung

Jeder neue Push auf den Source-Branch aktualisiert den MR. **MR ohne Merge schließen: Kein Code landet in `main`.**

MR-Beschreibung

```
## Änderung
Sortiert ListFiles.ps1 aufsteigend nach Namen.
Closes #12

## Nachweis
Test mit mehreren Dateinamen ergänzt.
Bytezahlen und Ausschluss von Verzeichnissen bleiben erhalten.

## Offener Punkt
Welche Sortierregeln sollen für Umlaute gelten?
```

Ein konkret genannter Test sagt mehr als „alles grün“. Die offene Frage steht sichtbar in der Beschreibung, nicht versteckt im Diff.

Rollen im Merge Request

Rolle im Review	Aufgabe
Autor	Erklärt und korrigiert die Änderung
Reviewer	Prüft Diff, Anforderung und Tests
Assignee	Ist für den MR zuständig
Person mit Merge-Recht	Führt den Merge aus

Die Rollen sagen, wer was tut. Wer als Reviewer eingetragen ist, bekommt dadurch **kein Merge-Recht.** Das regeln Projektrolle und Branch-Schutz.

Review-Kommentar und Suggestion

Kommentar zum Test:

> Der Test verwendet nur eine Datei. So fällt eine falsche Reihenfolge nicht auf. Bitte testen Sie mit zwei verschieden benannten Dateien und prüfen Sie die erste Zeile.

Der Kommentar sagt: **Was fällt auf → was folgt daraus → was soll geändert werden.**

Ein Suggestion-Block schlägt eine Codeänderung direkt vor. Ob sie stimmt, entscheidet trotzdem das Review.

Sortiertest mit zwei Dateinamen

Gekürzter Ausschnitt; `$listScript` und `$folder` sind schon angelegt:

```
[IO.File]::WriteAllText((Join-Path $folder 'beta.txt'), 'bb')
[IO.File]::WriteAllText((Join-Path $folder 'alpha.txt'), 'a')
$rows = @(& $listScript -Path $folder)
$rows | Should -HaveCount 2
$rows[0] | Should -Be 'alpha.txt - 1 Bytes'
$rows[1] | Should -Be 'beta.txt - 2 Bytes'
```

Mit **zwei verschiedenen Namen** fällt eine absteigende Sortierung auf. Wie Umlaute oder Groß-/Kleinschreibung sortiert werden, testen diese Namen noch nicht.

Review-Threads und Resolved

Der Reviewer fragt direkt an der Zeile. Der Autor antwortet oder ergänzt den Test. Dann wird geprüft: Ist die Frage beantwortet?

Resolved heißt: Die Diskussion ist erledigt. Es heißt nicht, dass der Code stimmt.

Mit der Einstellung **All threads must be resolved** blockieren offene Threads den Merge. Ohne diese Einstellung kann trotz offener Threads gemergt werden.

Approvals nach Tier

Mechanismus	Wirkung	Tier
Optionales Approval	Zustimmung, aber kein Muss	**Free**
Erforderliche Approval-Regel	Merge erst nach festgelegten Zustimmungen	**Premium / Ultimate**

Auf Free kann das Team Review **fest vereinbaren.** GitLab erzwingt das Approval dann aber nicht.

GitLab: Merge request approvals

Merge-Gates: Bedingung und Einstellung

Bedingung	Einstellung in GitLab
Diese Person darf mergen	Rolle und Branch-Schutz
Die Pipeline muss grün sein	**Pipelines must succeed**
Alle Diskussionen sind erledigt	**All threads must be resolved**
Der MR ist fertig, es folgen keine Commits mehr	Nicht mehr **Draft**, sondern **Ready**
Bestimmte Zustimmungen sind nötig	Approval-Regeln (je nach Lizenz)

Eine rote Pipeline blockiert den Merge nur, wenn das Projekt so eingestellt ist. Auch bei grünem Button prüfen Sie den **letzten Commit** inhaltlich selbst.

Merge-Methoden in GitLab

GitLab-Methode	Ergebnis
Merge commit	Eigener Merge-Commit
Merge commit with semi-linear history	Merge-Commit, nur wenn Fast-Forward möglich
Fast-forward merge	Zielbranch rückt ohne Merge-Commit vor

Alle drei gibt es in **Free**. Hat sich `main` weiterbewegt, müssen Sie eventuell aktualisieren oder rebasen. Keine Methode stellt sicher, dass **der zusammengeführte Stand** getestet wurde.

GitLab: Merge methods

Squash beim Merge

Angenommen, der Source-Branch enthält drei Arbeitscommits:

```
git log --oneline main..feature/dateiliste
```

```
e4b19c2 README präzisieren
91d07aa Sortiertest ergänzen
3c5f2e8 Implementierung sortieren
```

Squash fasst sie zu einem neuen Commit auf dem Zielbranch zusammen. In GitLab ist das der Haken **Squash commits** im MR. Ob noch ein Merge-Commit dazukommt, hängt von der Merge-Methode ab.

Squash lokal: `git merge --squash`

```
git switch main
git merge --squash feature/dateiliste
git status --short
git commit -m "feat: sortiere Dateiliste nach Namen"
git log --oneline -2
```

`merge --squash` übernimmt die Änderungen aller drei Commits in Working Tree und Index, legt aber **keinen Commit** an. Den schreiben Sie selbst mit einer Nachricht für die ganze Änderung.

Die drei Commits bleiben im Source-Branch erhalten. Für die nächste Änderung starten Sie am besten einen neuen Branch.

Reviewweg

1. Der MR erklärt: Warum, was genau, welche Tests?
2. Der Reviewer prüft und schreibt seine Kommentare.
3. Der Autor antwortet und passt die Änderung an.
4. Wer Merge-Recht hat, mergt, sobald die Merge-Checks erfüllt sind.

Eine Tabelle im README kann festhalten, **wer was prüft**. Das ersetzt kein Approval, das GitLab erzwingt.

Neuer Commit nach dem Review



- Der Reviewer prüft die Commits „Sortierung“ und „Test“ und gibt **Approve**.
- Danach pusht der Autor einen weiteren Commit in denselben Branch. Der MR zeigt ihn sofort mit an.
- Das Approve gilt nur für den geprüften Stand. Der neue Commit braucht **neues Review** und eine grüne Pipeline.

Rückblick Teil III

Die Branching-Strategie passt dazu, wie das Team mergt. Die Konfliktlösung erfüllt beide Anforderungen. Der MR erklärt die Änderung; Review und Tests beziehen sich auf **das, was gemergt wird**.

Als Nächstes lassen wir diese Prüfung **automatisch als Job laufen** und schauen auf seinen Status.

Teil IV · Die erste Pipeline

Lernziele Teil IV

- Pipeline, Job, Stage und Runner unterscheiden.
- Eine kleine `.gitlab-ci.yml` schreiben, die ein Testskript startet.
- Fehler über Status, Kommando und Log finden.
- Tests, ZIP und Merge-Freigabe getrennt prüfen.

`.gitlab-ci.yml` und der getestete Commit

GitLab liest `.gitlab-ci.yml` im Repository-Root. Die Datei legt Jobs und Ablauf fest; ein anderer Pfad ist einstellbar.

Was im Repository passiert	Was folgt
Commit A hat den alten Testaufruf	Die Pipeline testet genau Commit A
Commit B bringt den neuen Testaufruf	Neue Pipeline nötig
Branchname bleibt gleich	Der Commit kann trotzdem ein anderer sein

Eine grüne Pipeline gilt nur für ihren Commit. Für später gepushte sagt sie nichts.

Pipeline, Job, Stage, Runner, Executor

Begriff	Aufgabe
Pipeline	Ein Durchlauf aller Jobs der CI-Konfiguration
Job	Kommandos, endet grün oder rot
Stage	Fasst Jobs zusammen, legt ihre Reihenfolge fest
Runner	Nimmt passende Jobs an und führt sie aus
Executor	Legt fest, in welcher Umgebung Jobs laufen

Der Runner führt jeden Job einzeln aus, nicht die ganze Pipeline als ein Skript. Scope und Executor behandeln wir später.

Erster Job: `hello`

Ein Job für einen Runner, auf dem PowerShell installiert ist:

```
hello:
  script:
    - pwsh -NoProfile -Command 'Write-Output "COMMIT=$env:CI_COMMIT_SHORT_SHA"'
```

`hello` ist der Jobname, unter `script` steht das Kommando. `CI_COMMIT_SHORT_SHA` enthält die ersten 8 Zeichen der ID des Commits, der die Pipeline ausgelöst hat. `pwsh` schreibt sie ins Log.

Der Job zeigt nur: Der Runner läuft und kennt den Commit. Die Dateiliste testet er noch nicht.

`stages`: Reihenfolge der Jobs

Nur zur Struktur: Beide Jobs geben bloß Text aus.

```
stages: [test, package]
test:
  stage: test
  script: echo "Test-Schritt"
package:
  stage: package
  script: echo "Paket-Schritt"
```

Die Reihenfolge legt `stages` fest, nicht die Position der Jobs im YAML. Was ein Job wirklich tut, steht **in seinen Kommandos**.

Fehlgeschlagener Job und folgende Stages

Ergebnis	Was danach passiert
Test rot	Stage `package` startet nicht
Test grün	Stage `package` startet

Läuft in derselben Stage schon ein anderer Job, bricht GitLab ihn nicht ab.

Parallele Jobs derselben Stage

Zwei Testjobs in derselben Stage können parallel laufen. Ob sie wirklich gleichzeitig starten, hängt davon ab, ob passende Runner frei sind.

Jeder Job läuft in seiner eigenen Umgebung: Eine Datei aus einem Job ist **im nächsten Job nicht automatisch da**.

`image`: Job-Image beim Docker-Executor

```
default:
  image: gitlab.devadze.de:5050/schulung/ci-image:7.5
```

Beim Docker-Executor nennt `image` das Image, in dem der Job läuft. Darin müssen Git, PowerShell und alles für die Tests schon installiert sein.

Der Shell-Executor läuft dagegen direkt auf dem Host. Ein Image-Name **wählt keinen Executor und installiert keine Tools**. Auch einem „PowerShell-Image“ können Git und Module fehlen.

Shell des Containers und `pwsh`

Ausschnitt aus dem Job `test`:

```
test:
  stage: test
  script:
    - pwsh -NoLogo -NoProfile -File ci/Test.ps1
```

In unserem Linux-Container liest zuerst eine Shell die Zeile. Erst `pwsh` startet dann PowerShell und führt die `.ps1`-Datei aus.

Längere Testlogik gehört **ins Skript im Repository**. Kopieren Sie PowerShell-Befehle nicht ungeprüft ins YAML: Dort liest Bash mit.

Pester: Testframework für PowerShell

`ci/Test.ps1` führt die Tests mit **Pester** aus. Pester ist für PowerShell, was pytest für Python ist. Es ist im CI-Image installiert; lokal brauchen Sie es nicht.

```
Describe 'ListFiles' {
    It 'sortiert Dateien und lässt Unterverzeichnisse aus' {
        $rows = @(& $listScript -Path $folder)
        $rows[0] | Should -Be 'a.txt - 1 Bytes'
    }
}
```

- `Describe` fasst zusammengehörige Tests zusammen.
- `It` ist ein einzelner Test.
- `Should` prüft das Ergebnis. Stimmt es nicht, ist der Test rot.

Exit-Code des Testskripts

Der Runner sieht nur den Exit-Code des Prozesses, nicht die Testergebnisse.

Testlauf	Das Skript muss liefern
Alle Tests grün	Exit-Code `0`
Mindestens ein Test rot	Exit-Code ungleich `0`
Tests laufen nicht oder fehlen	Auch ungleich `0`

Die entscheidenden Zeilen aus unserem Testskript:

```
if ($run.TotalCount -eq 0 -or $run.Result -ne 'Passed') { exit 1 }
exit 0
```

Was ein Test findet und was nicht

Fehler im Code	Test
Falsche Sortierrichtung (`-Descending`)	**rot**: erste Zeile ist `beta.txt` statt `alpha.txt`
`Sort-Object` fehlt ganz	**oft grün**: das Dateisystem liefert die Namen schon sortiert

Ein grüner Test zeigt nur, dass das Geprüfte stimmt. Was der Test nicht prüft, kann trotzdem falsch sein.

`-ForceFailure`: absichtlicher Testfehler

Unser Testskript kann außerdem absichtlich scheitern:

```
test:
  stage: test
  script:
    - pwsh -NoProfile -File ci/Test.ps1 -ForceFailure
```

`-ForceFailure` lässt einen Test absichtlich scheitern. So sehen wir, ob die Kette hält: **Test rot → Exit-Code `1` → Job rot.**

Der rote Job heißt hier nicht, dass `ListFiles.ps1` falsch ist. Das klärt der Sortiertest: **Sortiert das Skript richtig?**

Joblog im Fehlerfall: Testergebnis und Exit-Code

Erwartete Logzeile beim absichtlichen Testfehler:

```
TESTS: total=5 passed=4 failed=1
```

Zusätzlich endet das Testskript mit Exit-Code `1`. Fehlt dieser Exit-Code, wird der Job grün, obwohl ein Test fehlschlug.

Lesen Sie beides: Was meldet das Testwerkzeug, und welchen Exit-Code sieht der Runner?

`artifacts`: Dateien aus dem Job `package`

Ausschnitt; das Skript `ci/Package.ps1` baut das ZIP:

```
package:
  stage: package
  script: pwsh -NoProfile -File ci/Package.ps1
  artifacts:
    paths: [dist/]
```

Das ZIP baut das Skript. `paths` sammelt nur ein, was danach im Projektverzeichnis liegt.

Das Artifact gehört zu diesem Joblauf. **Git-Tag, Historie und dauerhafte Ablage** brauchen eigene Schritte.

ZIP und Prüfsumme im Job `package`

`ci/Package.ps1` erzeugt zum Beispiel:

```
dist/banana-project-local.zip
dist/banana-project-local.zip.sha256
```

Die Prüfsumme passt nur zu **diesem ZIP**. Für ein später neu gebautes ZIP gilt sie nicht. Am Dateinamen allein erkennen Sie nicht, welches ZIP es ist.

Übergeben können Sie das Artifact erst, wenn die Dateien da sind und zum richtigen Commit gehören. Ist `dist/` leer, hilft auch ein grüner Job nichts.

Jobstatus `pending`, `running`, `failed`

Zustand	Was Sie prüfen
pending	Job wartet auf einen Runner; ist ein passender Runner frei?
running	Job läuft; im Log das aktuelle Kommando ansehen: Hängt es irgendwo?
failed	Job oder seine Vorbereitung ist gescheitert; erste echte Fehlermeldung im Log suchen

Zwei Fälle, in denen die Tests gar nicht laufen:

- Kein passender Runner frei: Der Job bleibt `pending`.
- Im Image fehlt `pwsh`: Der Job startet und scheitert sofort (`failed`).

Ein hängender oder roter Job heißt also nicht, dass ein Test rot ist. Erst im Log nachsehen, ob die Tests überhaupt gelaufen sind.

YAML-Einrückung von `script`

Richtig: `script` ist eingerückt und gehört zum Job `test`.

```
test:
  script:
    - echo "Prüfen"
```

Falsch: `script` steht auf derselben Ebene wie `test`. GitLab hält beide für eigene Jobs, und keiner hat ein `script`.

```
test:
script:
  - echo "Prüfen"
```

GitLab startet keine Pipeline und meldet:

`jobs test config should implement the script:, run:, or trigger: keyword`

Pipeline mit `test` und `package`

Setzt die Skripte `ci/Test.ps1` und `ci/Package.ps1` voraus:

```
stages: [test, package]
default:
  image: gitlab.devadze.de:5050/schulung/ci-image:7.5
test:
  stage: test
  script: pwsh -NoProfile -File ci/Test.ps1
package:
  stage: package
  script: pwsh -NoProfile -File ci/Package.ps1
  artifacts:
    paths: [dist/]
```

Erst Tests grün, dann ZIP bauen.

Joblog: Commit, Kommando, Fehlermeldung

Der Branchname reicht nicht: Nach jedem Push kann er auf einen anderen Commit zeigen.

Im roten Job	Sagt Ihnen
Commit	Welcher Code getestet wurde
Gestartetes Kommando	Was genau lief
Erste echte Fehlermeldung	Test rot, Tool fehlt oder Übertragung gescheitert?
Status	Wie der Job endete

Die letzte Logzeile meldet oft nur den Fehler. Die **Ursache** steht meist weiter oben.

Retry und neuer Commit nach der Korrektur

So soll es laufen (ohne `allow_failure`):

Commit	Test	ZIP bauen
Commit mit absichtlichem Fehler	Rot	Startet nicht
Neuer Commit mit Korrektur	Test läuft neu	Erst wenn Test grün

Ein Retry startet den alten Job mit dem alten Code. Die Korrektur muss als **neuer Commit** gepusht werden, damit eine neue Pipeline sie testet.

„Korrigiert“ heißt: Code geändert, Tests passen, neuer Lauf.

Merge-Bedingung „Pipelines must succeed“

Die Pipeline zeigt, ob ihre Jobs grün sind. Mit der Projekteinstellung **Pipelines must succeed** lässt GitLab den Merge erst zu, wenn die Pipeline grün ist.

Gibt es mehrere Pipelines, zählt oft die MR-Pipeline. Schauen Sie deshalb auf den Status im MR, nicht auf irgendeinen grünen oder roten Lauf.

Review, Branch-Rechte und erledigte Threads sind eigene Bedingungen. Ein grüner Test ersetzt kein Approval, ein Approval keinen Test.

Im Beispiel: Der Sortiertest muss auf dem letzten Commit gelaufen sein, und der MR muss den vereinbarten Merge-Regeln folgen.

Rückblick Teil IV

Konfiguration: Jobs und Stages legen einen klaren Ablauf fest. Ausführung: Ein passender Runner stellt die nötige Umgebung bereit.

Tests: Ein echter Fehler führt zu Exit-Code ungleich `0` und einem roten Job. Ergebnis: Ist der Job `package` grün, liegt das ZIP als Artifact vor.